DS 2500: Intermediate Programming with Data
Fall 2025

Homework 2: English Women's Soccer/Football Goals

Assigned: October 7, 2025
Deadline: October 21, 2025 at 11:59 PM ET
Total Points: 1,000 pts

Overview and Learning Objectives

In this homework, you will simulate a goal-scoring comparison among a bunch of English women's football (soccer) tier 1 Teams, taking place over the Barclays Women's Super League (WSL) 2023-2024 season. One line of the data file contains how many goals a given team scored in every game over the season. We want to compare the teams' total goals as they accumulated over time.

This assignment is intended to review the following concepts we covered in class:

- Classes and object-oriented programming.
- Data visualization.
- Statistics of Data Science (types of data, central tendency, etc.)
- Documenting code.

Submission Instructions

- Submit your solution as `.py` files (filenames are specified below) to DS2500 Gradescope by **11:59 PM ET October 21, 2025**.
- Late submission is allowed until **November 4, 2025** with a 100 point penalty. Submissions will not be accepted after the late submission deadline.
- You may submit to Gradescope as many times as you want to check your score and improve your score.

Additional Instructions

- Make sure to download the data file (csv) from **Canvas**. You are NOT given a starter code file for this assignment but are expected to create your own Python file(s).
    - You are free to use Google Colab for this assignment. However, you will **need to download your code as `.py` file for submission** and not as `.ipynb`
- We recommend using `Pandas` for data reading & manipulation, but it is not required. The only use of `Pandas` should be for reading the data file. Methods from this library cannot be used to calculate the mean, median, mode, standard deviation, etc. for your data.
- You may **NOT** use `statistics`, `NumPy`, or `SciPy` for this assignment. You can **only** use `csv` or `Pandas` and `matplotlib` libraries for this assignment.

- You are required to use `print()` inside the `main()` function or the main block/guard to test and understand your code.

Grading Policies

Homeworks are broken into three components:

1. **Auto grading (640 points):**
    a. **Accuracy**. You'll answer quantitative questions about the dataset on gradescope. These answers are auto-graded and not the code. Gradescope can be a little picky, so make sure you don't put extraneous characters or whitespace in your answers -- and double-check the "correct/incorrect" confirmation!
2. **Manual grading (360 points):** Details are in the DS2500 Grading Guidelines.
    a. **Visualization (90)**. You'll be asked to submit screenshots/downloads of your Python plot(s). We expect these plots to be labeled, easy to read and understand, and appropriate for the data.
    b. **Code Quality (270).** You'll submit your code as well, which we will review and grade based on its modularity, readability, and reusability.

Dataset

`wsl_goals_2324.csv` contains how many goals a given team scored in every game over the season.

Python Requirements

Define a `Team` class. Your class must have the following attributes, which are initialized in the constructor (you can add more attributes if you want, but don't leave out any of these):

- `name` The name of the team, a string.
- `goals` The number of goals scored in each game in 2023, a list of ints.
- `x` The team's x-position on the matplotlib grid.
- `y` The team's y-position on the matplotlib grid.
- `color` The team's color when rendered on matplotlib.

`name` is a required parameter for the constructor, `goals` is NOT a parameter, and the others are optional parameters. In your constructor, `goals` should be initialized to be an empty list. Use reasonable default values for the optional parameters such as `x = 0, y = 0, color = "purple"`.

In addition to the constructor, your Team class should also have the following methods (you can add more if you want):

- `draw(self)`. Render the team on the matplotlib grid at its current *x*, *y* position and with its color. You can make the team any matplotlib marker shape you want.
- `add_goals(self, goals)`. Append the given *goals*, an int, to the team's list of goals.

- `get_total_goals(self)`. Returns the total goals of all the goals in the *goals* list, an int.
- `move_next(self)`. Move the team to the right by the amount found in the next position in its *goals* list. The list method *pop* could be useful here!
- `__str__(self)`. Return a string that you'd like to be used when you call *print()* on a Team

Your solution file should contain a function named `main`. It should:

- Read in the given data file, where one line represents one team's data for the 2023-2024 season. Use `Pandas` for reading the file.
- For each line in the file, create a `Team` object, and keep all your Teams in a list.
- Generate the required plots and print answers to the questions below.

You can have a `team.py` file for defining your `Team` class and another file named `season.py` that imports the `team.py` file and contains the code for analyzing the data and generating the plots.

Part 1: Questions about the Data

This part of your solution will be auto-graded. When you find this assignment on Gradescope, the first part will ask you the following questions; type or select your answers. You must also compute the answers to these questions programmatically and **print them in your code**. Gradescope will confirm your answers are correct/incorrect. We expect everyone to get full credit on this part!

Check the output! Gradescope can be a little picky about formatting, and we don't want you to lose points for putting extra characters or whitespace in an answer. Make sure you've got the correct answer to each question for full credit.

Answer these questions in Gradescope:

1. Which team scored the most goals in season 2023-2024? **(25 points)**
2. What was the average (mean) of total goals scored by all the teams? **(25 points)**
3. What was the variance of total goals scored by all the teams? **(60 points)**
4. What was the standard deviation of total goals scored by all the teams? **(60 points)**
5. What was the median of goals scored in each game by *Arsenal*? **(60 points)**
6. What was the mode of goals scored in each game by *Tottenham Hotspur*? **(60 points)**
7. Which team has the most consistent goal-scoring pattern? **(100 points)**
    a. Calculate the coefficient of variation (standard deviation divided by mean) for each team and identify the team with the lowest value. The coefficient of variation gives you relative consistency i.e., how much a team's performance varies as a percentage of their typical performance. This is much more meaningful for comparing teams with different scoring capabilities as it is a normalized measure.

8. Which team had the most impressive scoring streak of *at least* 2 goals a game? **(150 points)**
    a. Find the longest consecutive streak of games where each team scored at least 2 goals in a game.
9. Which team improved their average goals per game the most in the second half? **(100 points)**
    a. Compare the first half of the season vs. second half performance for each team.

Part 2: Visualization

90 points: Submit one image of your plots. Your grader will run your code to see the full visualization animation.

When we run your code, your `matplotlib` code should:

- For each game of the season, generate a new plot that renders all the Team objects; update their x-values each time with the amount of goals they scored. (Matplotlib looks like a mini-animation when we render new plots over and over with slightly-different values. Here's our example.)
- Your plot should use colors that are similar to a color of the WSL team logo colors.
- Your plot should have a legend that displays the full team name ("Arsenal") OR an abbreviation of the team name based on below list:
    - ***Arsenal:*** *ARS*
    - ***Aston Villa:*** *AVL*
    - ***Brighton & Hove Albion:*** *BHA*
    - ***Bristol City:*** *BCFC*
    - ***Chelsea:*** *CHE*
    - ***Everton:*** *EFC*
    - ***Leicester City:*** *LCFC*
    - ***Liverpool:*** *LFC*
    - ***Manchester City:*** *MCFC*
    - ***Manchester United:*** *MUFC*
    - ***Tottenham Hotspur:*** *THFC*
    - ***West Ham United:*** *WHU*

Part 3: Code Quality

Submit on Gradescope the code you developed to compute your answers to the Part 1 questions and to generate the plots for Part 2. Your code will be graded on modularity, readability, and reusability as discussed in class and outlined in our grading guidelines.

Hints!

A few things that might be helpful...

- If a team didn't score any goals, it shows up in the `.csv` file as empty. Convert those to zeroes to ensure correct computations.

- For all variance and standard deviation calculations, use the *population formula* with `n` in the denominator and NOT the sample formula with `n-1`. This means that you should use the the following formulas (*first (left)* is for variance and *second (right)* is for standard deviation):

$$\frac{\Sigma(x-y)^2}{n} \quad \text{or} \quad \sqrt{\frac{\Sigma(x-y)^2}{n}}$$

  where

  Σ = sum of…

  y = population mean

  n = number of scores in sample
- After creating your list of `Team` objects, you can use the `get_total_goals` method to help determine your `xlim` values.
- Your other methods will help to update each team's position on the plot and generate your visualizations
- To generate new plots without needing to click out of the window, you might find these helpful: `plt.pause()`, `plt.show(block = False)`, `plt.figure()`.

Tips for Success

1. **Start Early:** This assignment covers many fundamental concepts - give yourself plenty of time
2. **Test Incrementally:** Test each function as you write it
3. **Read Instructions Carefully:** Pay attention to which built-in functions you can and cannot use
4. **Use Helper Functions:** Break complex problems into smaller, manageable pieces if needed
5. **Comment Your Code:** Write clear comments explaining your logic
6. **30-minute Guideline:** If you get stuck on a homework problem, come by office hours or post on Piazza! We recommend you spend about 30 minutes trying to figure out a problem, and then ask for help. This will be enough time that you can try a few things to get unstuck, but not SO much time that you're banging your head against the wall. Try for 30 minutes, then ask us.